

Exercise 01 (Linked List - Sum of Values):

Create a linked list containing the values: 6, 4, 10, 9, 1

Write a method `sum_values()` that returns the sum of all values.

```
from collections import deque
class MyLinkedList:
    def __init__(self):
        self.list = deque()
    def sum_values(self):
        total = 0
        for value in self.list:
            total += value
        return total
obj = MyLinkedList()
obj.list.append(6)
obj.list.append(4)
obj.list.append(10)
obj.list.append(9)
obj.list.append(1)
print("Sum =", obj.sum_values())
```

Output:

```
Sum = 30
```

Exercise 02(Linked List Parameter):

Write a function `sum_numbers()` that receives a linked list as a parameter and returns the sum of its values.

```
from collections import deque
def sum_numbers(lst):
    total = 0
    for number in lst:
        total += number
    return total
numbers = deque([6, 4, 10, 9, 1])
print("Sum =", sum_numbers(numbers))
```

Output:

```
Sum = 30
```

Exercise 03 (Stack - Balanced Parentheses):

Write a function `check(expression)` that checks whether the parentheses in an expression are balanced using a Stack.

Write a test program to demonstrate the function.

```
def check(expression):
    stack = []
    for ch in expression:
        if ch == "(":
            stack.append(ch)
        elif ch == ")":
            if not stack:
                return False
            stack.pop()
    return len(stack) == 0
expression = input("Enter expression: ")
if check(expression):
    print("Balanced expression")
else:
    print("Not balanced expression")
```

Output 1:

```
Enter expression: (((a+b)*c)+(u*v))
Balanced expression
```

Output 2:

```
Enter expression: (a+b)*c)
Not balanced expression
```

Exercise 04 (Queue):

In a bank, customers wait in a queue (q0) for their turn to complete a transaction.

Write a method `split_queue(q0, q1, q2)` that splits the customers into two queues:

q1: Names starting with A–H

q2: Names starting with I–Z

Write a test program that reads customer names, fills q0, calls the method, and displays the contents of q1 and q2.

```
from collections import deque
def split_queue(q0, q1, q2):
    while q0:
        name = q0.popleft()
        first_letter = name[0].upper()
        if "A" <= first_letter <= "H":
            q1.append(name)
        else:
            q2.append(name)
    q0 = deque()
    q1 = deque()
    q2 = deque()
    n = int(input("Enter number of customers: "))
    for i in range(n):
        name = input("Enter customer name: ")
        q0.append(name)
    split_queue(q0, q1, q2)
    print("\nQueue 1 (A-H):")
    while q1:
        print(q1.popleft())
    print("\nQueue 2 (I-Z):")
    while q2:
        print(q2.popleft())
```

Output:

```
Enter number of customers: 6
Enter customer name: Ali
Enter customer name: Sara
Enter customer name: Hassan
Enter customer name: Omar
Enter customer name: Ahmad
Enter customer name: Lina
```

Queue 1 (A-H):

```
Ali
Hassan
Ahmad
```

Queue 2 (I-Z):

```
Sara
Omar
Lina
```

Exercise 05 (Big-O):

Prove that: $n^3 + 1000 = O(n^3)$

According to the definition of Big-O:

$$f(n) = n^3 + 1000 \quad g(n) = n^3$$

We need constants c and n_0 such that: $f(n) \leq c \times g(n)$

Choose: $c = 2$ $n_0 = 10$ For every $n \geq 10$:

$$n^3 \geq 1000$$

$$\text{Therefore: } n^3 + 1000 \leq n^3 + n^3$$

$$\text{So: } n^3 + 1000 \leq 2n^3$$

$$\text{Hence: } f(n) \leq c \times g(n)$$

$$\text{Therefore: } n^3 + 1000 = O(n^3)$$

Final Answer: $c = 2$ $n_0 = 10$

Exercise 06 (Big-O Analysis):

Determine the running time of the following code using Big-O notation.

Step 1: Create the array `int[] a = new int[n]` --> $O(1)$

Step 2: First loop for `(i = 0; i < n; i++)` --> $O(n)$

Step 3: Second loop for `(i = 0; i < n; i++)` --> $O(n)$

Step 4: Total Running Time $O(1) \cdot O(n) \cdot O(n) = O(n^2)$

Exercise 07 (Recursion):

Write a recursive method named `odd_sum()` that takes a positive odd integer n and returns the sum of odd integers from 1 to n .

Example:

$$\text{odd_sum}(5) = 1 + 3 + 5 = 9$$

```
def odd_sum(n):
    if n == 1:
        return 1
    return n + odd_sum(n - 2)
n = int(input("Enter an odd number: "))
print("Sum =", odd_sum(n))
```

Output:

```
Enter an odd number: 7
Sum = 16
```

Exercise 08 (Recursive oddSum) :

Write a recursive method named `odd_sum()` that takes a positive odd integer `n` and returns the sum of odd integers from 1 to `n`.

Example:

`odd_sum(7) = 1 + 3 + 5 + 7 = 16`

```
def odd_sum(n):
    print("odd_sum(", n, ")", sep="")
    if n == 1:
        return 1
    return n + odd_sum(n - 2)
n = int(input("Enter a positive odd number: "))
print("Sum of odd numbers =", odd_sum(n))
```

Output:

```
Enter a positive odd number: 9
odd_sum(9)
odd_sum(7)
odd_sum(5)
odd_sum(3)
odd_sum(1)
Sum of odd numbers = 25
```

Exercise 09 (Recursion - Palindrome):

A palindrome is a word that reads the same forwards and backwards.

Write a recursive method named `is_palindrome()` that takes a string as a parameter and returns `True` if the word is a palindrome and `False` otherwise.

Write a test program to test the method.

```
def is_palindrome(word):
    if len(word) <= 1:
        return True
    if word[0] != word[-1]:
        return False
    return is_palindrome(word[1:-1])
word = input("Enter a word: ")
if is_palindrome(word):
    print("The word is a palindrome")
else:
    print("The word is not a palindrome")
```

Output 1:

```
Enter a word: otto
The word is a palindrome
```

Output 2:

```
Enter a word: hello
The word is not a palindrome
```

Exercise 10 (Tree & Recursion):

Write a recursive method that checks whether a binary tree is a full binary tree.

A full binary tree is a tree in which every node has either 0 or 2 children.

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
def is_full_binary_tree(root):
    if root is None:
        return True
    if root.left is None and root.right is None:
        return True
    if root.left is None or root.right is None:
        return False
    return (is_full_binary_tree(root.left) and
            is_full_binary_tree(root.right))
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)
root.right.right = TreeNode(7)
if is_full_binary_tree(root):
    print("The tree is a full binary tree")
else:
    print("The tree is not a full binary tree")
```

Output:

The tree is a full binary tree

Example of NOT Full Binary Tree:

```
  1
 /\
2  3
 /
4
```

Output:

The tree is not a full binary tree

Exercise 11 (Graphs - DFS):

Create a graph using an adjacency list and implement Depth First Search (DFS) to visit all vertices.

```
class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.adjacency_list = [[] for _ in range(vertices)]
    def add_edge(self, source, destination):
        self.adjacency_list[source].append(destination)
    def dfs(self, start):
        visited = [False] * self.vertices
        self._dfs_util(start, visited)
    def _dfs_util(self, vertex, visited):
        visited[vertex] = True
        print(vertex, end=" ")
        for neighbor in self.adjacency_list[vertex]:
            if not visited[neighbor]:
                self._dfs_util(neighbor, visited)
graph = Graph(5)
graph.add_edge(0, 1)
graph.add_edge(0, 2)
graph.add_edge(1, 3)
graph.add_edge(2, 4)
print("DFS Traversal:")
graph.dfs(0)
```

Output:

```
DFS Traversal:
0 1 3 2 4
```

Exercise 12 (Dictionary):

Create a program that stores student names and their grades using a dictionary, then searches for a student's grade.

```
students = {
    "Ali": 90,
    "Ahmad": 85,
    "Sara": 95
}
print("Students Grades:")
print(students)
name = input("Enter student name: ")
if name in students:
    print("Grade =", students[name])
else:
    print("Student not found")
```

Output:

```
Students Grades:
{'Ali': 90, 'Ahmad': 85, 'Sara': 95}
Enter student name: Sara
Grade = 95
```

Exercise 13 (List):

Create a list that stores numbers. Add elements, remove an element, and display the list.

```
numbers = []
numbers.append(10)
numbers.append(20)
numbers.append(30)
numbers.append(40)
print("List:", numbers)
numbers.pop(2)
print("After removing element:", numbers)
print("First element:", numbers[0])
```

Output:

```
List: [10, 20, 30, 40]
After removing element: [10, 20, 40]
First element: 10
```